NAVIKA

Student-aware school & college bus tracking — for India, on a phone, with zero hardware.

Solo-built full-stack product · a dot on a map is not an answer — a child on a bus is

Next.js 16 · React 19 · TypeScript 5 · Tailwind v4
Prisma 7 · PostgreSQL 17 · Socket.IO 4.8 · jose
MapLibre GL 6 · OSRM · Web Push · Zod v4 · Vitest 5

83% of the 6-phase roadmap is built and running. Phases 0–4 shipped, Phase 5 started

147/147 unit tests passing; typecheck and ESLint clean. GPS and ETA maths is tested, not eyeballed

18 database models with real foreign keys — no orphans, no `relationMode` escape hatch

₹0 hardware cost per bus. The driver's phone is the tracker — no black box, no RFID cards

01 KIS CHEEZ KE LIYE — THE PROBLEM

At 7:40 in the morning a parent does not ask "where is the bus?" They ask "is my child on it?"

Navika closes the gap between **where a vehicle is** and **what happened to a person** — which student is on which bus, which stop they board at, whether they have stepped out of the house yet, and whether they actually got picked up.

1. **The hardware gate**
   An affordable Indian school buys a ₹3,000–8,000 GPS black box per bus, RFID cards, a SIM and an annual contract — and still receives a dot on a map. Cost scales per vehicle, so small colleges are priced out entirely.
2. **Nobody closes the loop**
   The bus arrives, the driver is late, the child is left standing. No product makes the parent and the driver **agree on what happened**. The map is a one-way broadcast with no confirmation channel.
3. **Identity is lost**
   A bus is tracked, a child is not. "Bus 12 is on time" never becomes "Aarav got home", and attendance becomes a register the driver ticks from memory after the route ends.

02 ISKO KAUN USE KAREGA

4 roles, role-gated at the query layer

Parent / Guardian primary user · daily
Live map of their child's bus, road-following ETA to **their** stop, one-tap "I'm waiting", pick-up and drop confirmations, web push when the app is closed. The reason the product exists.

Student ages 6–18
Own bus and ETA, their own waiting signal, a personal attendance history. The waiting button is the one action a child can genuinely take for themselves.

Driver / Conductor field user · hours per trip
Installable phone PWA using `watchPosition`, a live pick-up checklist showing who is waiting at the next stop, one-tap picked / missed. Built for a cheap Android, not a desktop.

Admin / Transport Operator the paying buyer
Stops, routes, buses, drivers, enrolments, trip scheduling, a live fleet map, emergency alerts and a 30-day attendance report — the whole operation in one portal.

03 KITNA KAAM HUA

Phases 0–2 committed · 3–4 in tree

Phase 0 — Foundation	18-model schema, session auth, 4 role dashboards, Socket.IO	DONE	100%
Phase 1 — Admin CRUD	Stops, routes, buses, drivers, enrolments, trips, attendance	DONE	100%
Phase 2 — Live Maps	MapLibre canvas, route polyline, stop markers, OSRM ETA	DONE	100%
Phase 3 — Driver PWA	GPS fix validation, driver panel, installable service worker	WIP	90%
Phase 4 — Waiting Sync	"I'm waiting", driver check-off, notifications, web push	WIP	90%
Phase 5 — Ship It	CSV export, offline queue, PWA polish, staging deploy	TODO	20%

Precisely what is unfinished. The `isSimulated` flag exists and two live trips are seeded, but there is no driver-facing simulator screen — needed to demo and test without draining a phone. Pick-up records are written and a 30-day report renders, but linking every enrolled student to a trip without the driver opening the checklist is not done.

How the build stays honest. `npm run verify` chains typecheck, ESLint, 147 Vitest cases and a production build, so a broken commit cannot land. Three smoke suites guard the realtime layer: `smoke` refuses an anonymous socket and a forged cookie, `smoke:auth` logs in as all four demo roles, `smoke:admin` walks the admin surfaces.

04 HUM KAISE ALAG HAIN

Already in the code, not promised

1. **Zero hardware — the phone is the tracker**
   No black box, no RFID, no SIM, no installer on campus. Per-bus cost is ₹0, so a 20-bus college onboards in a day, not a quarter. This is the single biggest reason incumbents cannot reach Navika's segment.
2. **The child is the entity, not the bus**
   Student → bus → stop → route → trip, and alerts resolve through `ParentChild` to that specific child's guardians. "Bus 12 is on time" becomes "Aarav got home." Fleet tools structurally cannot do this.
3. **The loop actually closes**
   Parent presses "I'm waiting" → the driver's checklist shows "3 waiting" before he arrives → he marks picked or missed. That two-way handshake is the proof a map can never give, and it is built end to end.
4. **GPS is treated as untrusted input**
   Accuracy ceiling, clock-skew window, a 160 km/h teleport guard and a freshness tone so a dead signal never looks like a parked bus. Most trackers draw the glitch and lose the parent's trust in week one.
5. **Free maps and free ETA at school scale**
   MapLibre + OpenStreetMap, OSRM with a haversine fallback. Vendors hide a per-seat Google bill in the annual quote; here the marginal cost of one more student is zero.
6. **Operationally complete, not a demo**
   Admin CRUD, trip scheduling, live fleet map, emergency alerts and a 30-day attendance report ship alongside the tracking, on a ₹0–2k/month Node host. One persistent server is the price of live tracking, and it is a cheap one.

05 TECH STACK — AUR KYUN WO WALA

LAYER	CHOICE, AND THE REASON
Framework	Next.js 16.3 App Router, React 19, Turbopack — Server Components for data-heavy dashboards, one deploy target instead of a separate API app
Database	PostgreSQL 17 + Prisma 7.10 via the `pg` adapter, 18 models — real foreign keys and transactions; Prisma's `relationMode="prisma"` quietly abandons integrity on Postgres
Auth	`jose` JWT in an httpOnly, SameSite=Lax cookie — nothing in `localStorage` for an XSS bug to steal. Registration can only ever create STUDENT or PARENT
Authorization	A DAL beside every query — `proxy.ts` only redirects for UX. A redirect is a nicety, not a security boundary
Realtime	Socket.IO 4.8 on a custom Node server with per-bus rooms — the handshake reuses the session cookie, so there is no second token. Broadcast every ping, persist at most every 15 s
Maps	MapLibre GL 6 + OpenStreetMap — no API key, no per-seat bill, no Google lock-in
ETA	OSRM road-following API with a haversine fallback — accurate when the service is up, degrades to straight-line rather than failing
GPS trust	Accuracy ceiling, clock-skew window, 160 km/h teleport guard — a phone in a basement reports 2 km accuracy and a tunnel re-acquisition implies 300 km/h; drawing that glitch teleports a bus across a parent's screen
PWA	Service worker + web-push with VAPID — keys are optional, so the core still runs with no push service, losing only lock-screen alerts
Validation	Zod v4 — one schema imported by both the form and the Server Action, so the client is never trusted to match the server
Testing	Vitest 5, 147 cases — every decision about a position report is a pure function, so the hard cases are unit-tested rather than demoed
Seed data	3 routes, 12 stops, 5 buses, 6 drivers, 40 students, 8 parents, two live trips — a believable dataset, so a reviewer sees a product, not an empty shell
Hosting	A Node host — Render, Railway, Fly or a ₹500 VPS. Deliberately not Vercel: Socket.IO needs a persistent connection that serverless will not hold

06 KYA AISA ALREADY BANA HAI?

Yes — crowded market

India — hardware suites
Tracko · Campus Go · BusTrak · myskoolbus · ParentsEye · TrackYatra
GPS black box + RFID card attendance + parent app, sold as an annual per-vehicle or per-student contract. **The gap:** a one-way broadcast. No waiting signal, no confirmed pick-up, and the per-vehicle cost locks small colleges out.

India — light / parent-only apps
Kidobus · bustrackr · schoolites
Live map, push alerts, stop-level ETA, often free to the school to drive adoption. **The gap:** a map with a bell on it — no attendance proof, no route administration, no emergency handling for the operator.

US / Canada — district platforms
TrackSchoolBus · VeriTrans · BusBoss · Transfinder · Busology
AVL telemetry, route optimisation, district compliance reporting, RFID rider tracking. **The gap:** built for a district with a procurement department, not one college in one city. Six-figure contracts.

Generic fleet tooling
Google Fleet · Maps · standalone GPS trackers
Vehicle position, telematics, route history, engine diagnostics. **The gap:** no child identity, no stop, no parent. A fleet tool answers "where is asset 12" and never "is Aarav on it".

07 HONESTLY, KYA BAKI HAI

Not built yet, and it matters: CSV / Excel export of attendance and reports · an offline-first queue so a driver in a dead zone does not lose pick-ups · the driver-facing simulator · invite-vetted parent self-signup instead of open registration · billing and per-school tenancy · a hardware fallback for buses where no driver carries a phone · offline map tiles for poor-network corridors.

Why not Vercel. Socket.IO needs one long-lived connection to fan positions out to watching parents, and serverless functions are torn down between requests. Navika therefore runs on a persistent Node host behind a custom `server.ts` that hands the handshake to engine.io before Next.js sees the request.

08 AGLE 90 DIN — THE PLAN

Next 30 days	Close Phase 5. CSV / Excel export of attendance and reports, an offline-first queue so a driver in a dead zone does not lose pick-ups, the driver-facing simulator, and a staging deploy to a Railway or Render Node host with real VAPID keys so push is live rather than optional.
Days 30–60	Make it sellable. Invite-vetted parent self-signup instead of open registration, per-school tenancy and billing, offline map tiles for poor-network corridors, and onboarding documentation aimed at one school operator rather than a developer.
Days 60–90	Prove the metric. Three real school pilots live, with "was the pick-up confirmed" as the north-star number instead of daily actives. That result decides whether the defensible moat is the zero-hardware cost or the attendance history that accumulates on top of it.

How to see it working today. `npm run dev` serves the app on `localhost:3000` with Socket.IO attached. Seeded demo logins, all with the password `navika123`: `admin@navika.test`, `driver1@navika.test`, `parent1@navika.test` and `aarav.sharma0@student.navika.test` — one of each role, so the four dashboards can be compared side by side.